

The image features the text "DL for ENG" in a bold, 3D, gold-colored font. The letters are floating on a vast, blue ocean with visible ripples and small waves. In the background, there are rolling blue hills or mountains under a sky filled with soft, grey clouds. The overall scene has a serene and expansive feel.

DL for ENG

LECTURE 7.1

Physics-Informed Neural Networks

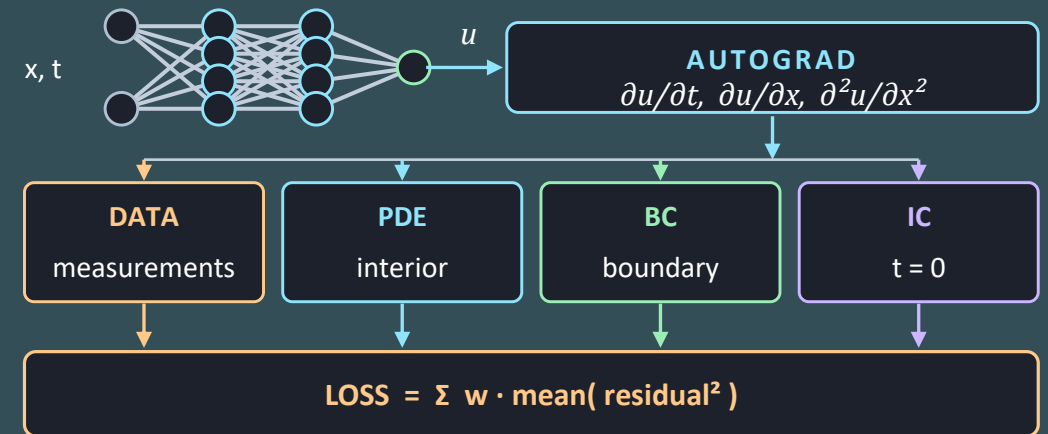
Remus Teodorescu

With support from Research Assistant Noman Khan -
nomank@energy.aau.dk

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026

ANSATZ · RESIDUAL · CONDITIONS



a gradient step on the weights, then again

A physics-informed network is a dense network that is the solution of a differential equation: autograd gives its derivatives, and the loss is the equation's residual at sampled points plus its boundary and initial conditions.

Learning Objectives

SECTIONS, IN ORDER

- 1 **PINN** — what it is, the solution ansatz, smooth activations
- 2 **Formulation** — the operator form, boundary and initial conditions, forward and inverse
- 3 **Loss** — the composite loss, soft and hard conditions, weights, collocation

Autograd and the optimisers, Adam then L-BFGS, are L6.1's: this lecture uses them without teaching them again.

READ, AND WHAT EACH ANSWERS

- | | |
|--------------------------|--|
| Liu ch. 2.1 | what is a PINN, and what can it represent? |
| Liu ch. 2.2 | which equation, conditions and unknowns? |
| Liu ch. 2.2 – 2.9 | how does each condition enter the loss? |

Recap

1 – 5 · PRE-TRAINING AND TRANSFER

- 1 What is transfer learning, and why do the first layers of a trained network carry over to a new task? *UDL ch. 9.3.6*
- 2 What does a transferred network compute: which layers are kept, which are replaced, and why? *UDL ch. 9.3.6; Ex_06 notebook 03*
- 3 How is it fine-tuned: what do you unfreeze, at what learning rate, and what is catastrophic forgetting? *UDL ch. 9.3.6, 12.6; Ex_06 notebook 03*
- 4 How did transfer compare with training from scratch in notebook 03, and when would it stop helping? *Ex_06 notebook 03*
- 5 How is the model you start from made, and what does continued pre-training on your own unlabelled data add? *UDL ch. 9.3.7*

6 – 10 · REINFORCEMENT TRAINING

- 6 What kind of problem is reinforcement learning, and how do state, action, policy and reward map onto control? *UDL ch. 19.1*
- 7 What does the policy compute, what is the return it maximises, and why discount it by γ ? *UDL ch. 19.1 – 19.2*
- 8 How is a policy trained: how does the policy gradient get a gradient from a world you cannot differentiate? *UDL ch. 19.5*
- 9 How does reinforcement learning differ from supervised learning, what is credit assignment, and where does RL not fit? *UDL ch. 19*
- 10 What is it used for in energy, and when is imitating an expert, such as an MPC, the better choice? *UDL ch. 19.7*

Physics-Informed Network

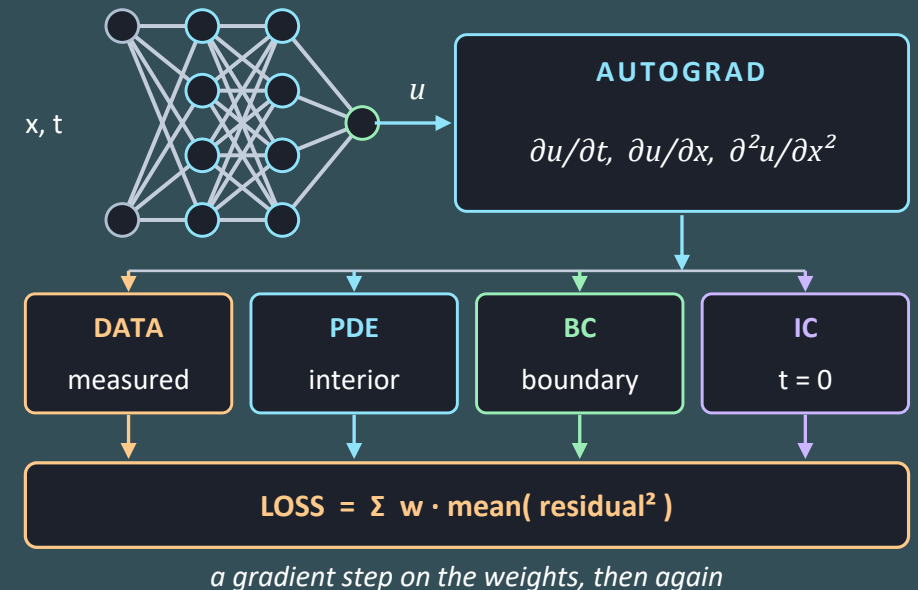
Physics-informed network — a neural network trained to be the solution of a differential equation: it takes the coordinates x and t and outputs the field u there. Its loss is the equation itself: autograd gives the derivatives of u , and the residual of the equation, squared and averaged over sampled points, is minimised together with the boundary and initial conditions.

AGAINST A MESH-BASED SOLVER

	FEM, FD	PINN
the unknowns	nodal values	the network's weights
the derivatives	a stencil or an element	autograd, exact
the geometry	meshed	sampled

TALK · 2 MIN How would you explain what a physics-informed network is, in your own words? Then, in the figure on the right, where do the equation and its conditions enter, and what does the data term add?

ONE NETWORK, ONE LOSS



No labels are needed: the equation and its conditions are the targets.

Solution Ansatz

Solution ansatz — the network written as the approximate solution, $u(x, t) \approx \mathcal{N}(x, t; \theta)$. A wide enough network can come as close as needed to any continuous solution; training chooses θ , and how many points it needs is the Collocation slide's question.

THE NETWORK IS THE SOLUTION

$$u(\mathbf{x}, t) = \mathcal{N}_{\theta}(\mathbf{x}, t)$$

$\mathcal{N}$ the network, θ its parameters, u the field it stands for

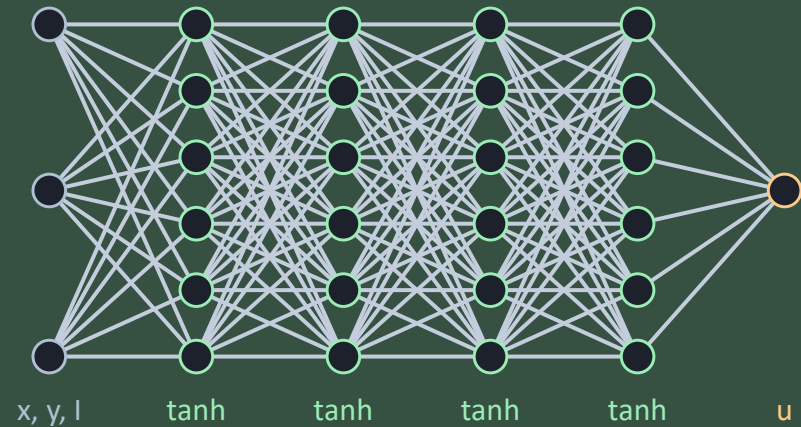
- **one network** — u at any point, no interpolation
- **no mesh** — the weights are the unknowns

THE ANSATZ IN PYTORCH

```
def u(x, y, I):
    return net(torch.stack([x, y, I], 1))
u_probe = u(x0, y0, I0)
```

the solution ansatz
the network is u
u anywhere, no mesh

EX_07.1: FOUR LAYERS OF 32



inputs x, y and the current I

hidden 4 layers of 32 neurons, $\tanh$

output u , the field at that point

3 329 weights, shared by every point: they are what training solves for.

Smooth Activation Functions

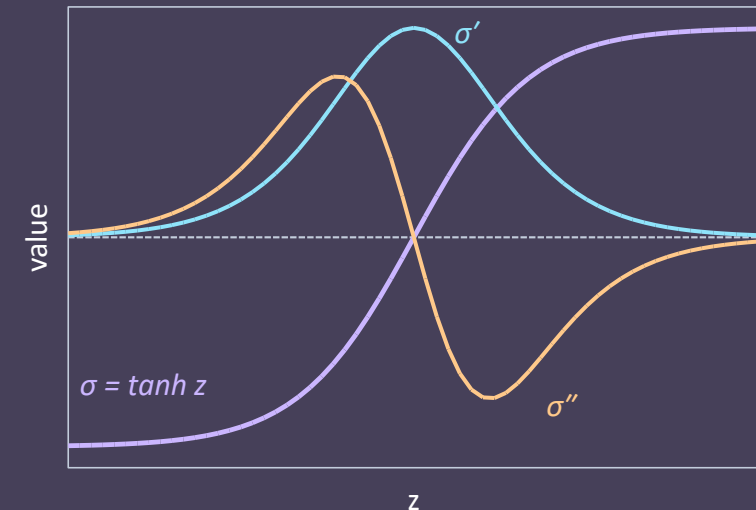
Smooth activation functions — without one, the layers collapse into one linear function, $u = \mathbf{a} \cdot \mathbf{x} + b$: a flat plane with $\nabla^2 u = 0$, which cannot bend into a hot spot or a wave. The activation gives the network curvature; a residual with k -th derivatives needs its k -th derivative to exist and not be zero. $\tanh$ is smooth to every order.

TANH AND WHAT THE RESIDUAL USES

	of $\tanh$	the residual uses it for
$\sigma(z)$	$\tanh z$	the field u itself
$\sigma'(z)$	$1 - \tanh^2 z$	first order: u_t, u_x
$\sigma''(z)$	$-2 \tanh z (1 - \tanh^2 z)$	second order: u_{xx}, u_{tt}

- **the order of the equation** — sets the smoothness
- **tanh** — the activation of every Part 2 network
- **sine** — one exception, for waves, in L7.2

TANH AND ITS FIRST TWO DERIVATIVES



All three are smooth. Amber is $\tanh$'s second derivative: the curve a Laplacian in the residual sees.

Operator Form

Operator form — the equation written as $\mathcal{F}[u] = 0$ in the domain Ω : the differential operator $\mathcal{F}$ is what the equation does to the true field u , with physical parameters c . The residual $r = \mathcal{F}[\mathcal{N}]$ is the same operator applied to the network instead; training drives it to zero.

ONE EQUATION FOR EVERY PROBLEM IN PART 2

$$\mathcal{F}(u; \mathbf{x}, t, c) = 0, \quad \mathbf{x} \in \Omega, \quad t \in [0, t_{\text{end}}]$$

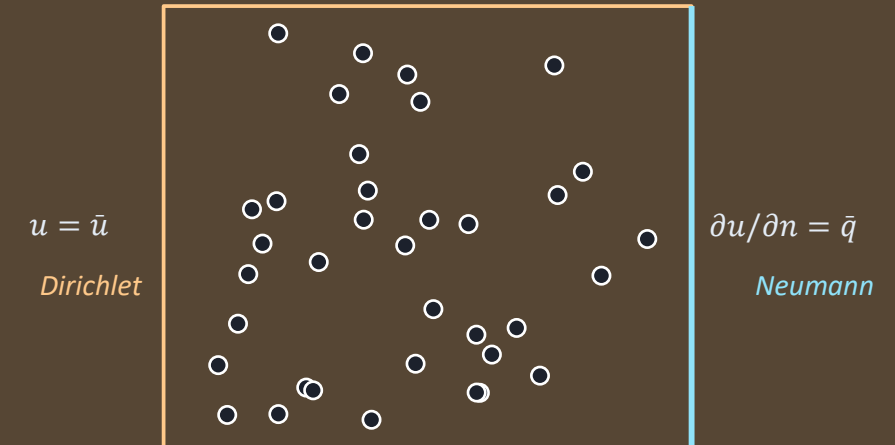
equation $\mathcal{F}[u] = u_{xx} + u_{yy} + f(x, y)$
Poisson; $f(x, y)$ is the source, for the slot its heat over k

residual $r(\mathbf{x}) = \mathcal{F}[\mathcal{N}_\theta](\mathbf{x})$
the network in place of u : zero where it solves the equation

THE RESIDUAL IN PYTORCH

```
xy.requires_grad_(True); u = net(xy)           # the network at the points
du = torch.autograd.grad(u.sum(), xy, create_graph=True)[0]  # ∇u
u_xx = torch.autograd.grad(du[:, 0].sum(), xy, create_graph=True)[0][:, 0]
r = u_xx + u_yy + f                           # the residual; u_yy likewise
```

THE DOMAIN AND ITS BOUNDARY



n is the outward normal: $\partial u / \partial n$ is the change across the wall, the flux. $\bar{u}$ is a given value, $\bar{q}$ a given flux; $\bar{q} = 0$ is an insulated wall.

Boundary and Initial Conditions

Boundary and initial conditions — the extra equations that make a problem well posed: a Dirichlet condition fixes the value on part of the boundary, a Neumann condition its normal derivative, an initial condition the field at $t = 0$, and an initial velocity its rate at $t = 0$ when the equation is second order in time.

FOUR CONDITIONS, FOUR RESIDUALS

DBC $u(\mathbf{x}, t) = \bar{u}(\mathbf{x}, t) \quad \forall \mathbf{x} \in \partial\Omega_{DBC}$

NBC $\mathcal{B}(\mathbf{x}, t, c) = 0 \quad \forall \mathbf{x} \in \partial\Omega_{NBC}$

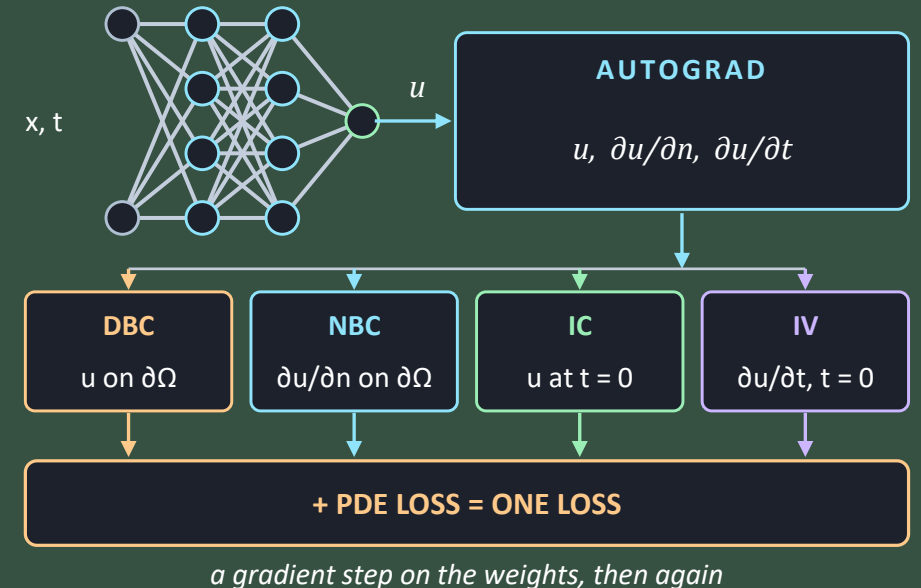
IC $\mathcal{I}(\mathbf{x}, 0, c) = 0 \quad \forall \mathbf{x} \in \Omega$

IV $\mathcal{V}(\mathbf{x}, 0, c) = 0 \quad \forall \mathbf{x} \in \Omega$

THE CONDITIONS IN PYTORCH

```
B_d = net(x_wall) - u_bar           # Dirichlet: the value
B_n = (grad(u, x_flux) * n).sum(1) - q_bar  # Neumann: the flux  $\partial u / \partial n$ 
I = net(x_t0) - u_0                 # initial: u at  $t = 0$ 
V = grad(u, x_t0)[: , -1:] - v_0     # velocity:  $\partial u / \partial t$  at  $t = 0$ 
```

WHERE EACH CONDITION ENTERS



Poisson: walls only · heat: walls and IC · wave: walls, IC and IV.

Forward and Inverse Problems

Forward and inverse problems — in a forward problem the equation and its parameters are known and the network finds the field. In an inverse problem some parameters c are unknown: they become trainable beside the weights, and sparse measurements, added to the loss as a data term, identify them.

TWO SETTINGS, ONE LOOP

$$u_t + \mathcal{F}[u; c] = 0$$

a time derivative plus the operator with parameters c ; Raissi writes $N[u; \lambda]$

$$\mathcal{L}(\theta, c) = \overline{r(\mathbf{x}; c)^2} + w_d \overline{(\mathcal{N}_\theta(\mathbf{x}_s) - u_s)^2}$$

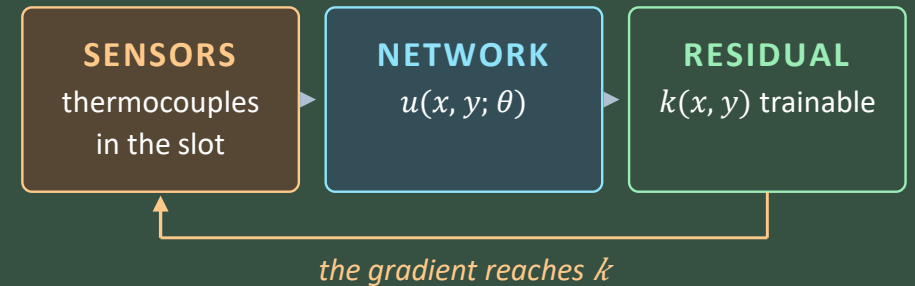
residual with c inside, plus the misfit at the sensors; minimised over ϑ and c

AN INVERSE PROBLEM IN PYTORCH

```
c = nn.Parameter(torch.tensor(1.0))          # c, the unknown parameter
opt = torch.optim.Adam([*net.parameters(), c]) # trained beside  $\theta$ 
res = r(net, x, c).pow(2).mean()               # residual, c inside
loss = res + w_d * (net(x_s) - u_s).pow(2).mean() # + data at the sensors
```

THE TWO SETTINGS

	forward	inverse
given	$\mathcal{F}, c$, conditions	$\mathcal{F}, u$ at a few sensors
unknown	u	u and c
data term	absent	essential
cost	one run per c	one run
example	new slot: k known	aged slot: find k



One run: the temperature and the aged insulation.

Composite Loss

Composite loss — the sum of one mean squared residual per equation: the PDE on interior points, each condition on its own boundary or initial points, and the measurements if there are any, each multiplied by a weight. Every term vanishes at the exact solution.

SIX TERMS, EACH A MEAN SQUARED RESIDUAL

$$\mathcal{L} = \mathcal{L}_{data} + w_{PDE}\mathcal{L}_{PDE} + w_{DBC}\mathcal{L}_{DBC} + w_{NBC}\mathcal{L}_{NBC} + w_{IC}\mathcal{L}_{IC} + w_{IV}\mathcal{L}_{IV}$$

$$\mathcal{L}_{PDE} = \frac{1}{n_{PDE}} \sum_{i=1}^{n_{PDE}} [\mathcal{F}_{\theta}(\mathbf{x}_i, t_i)]^2$$

the PDE term: the residual, squared, averaged over n interior points

THE COMPOSITE LOSS IN PYTORCH

```
L_pde = F.res.pow(2).mean()           # interior points
L_dbc = B_d.pow(2).mean(); L_ic = I.pow(2).mean() # their own points
loss = w_pde * L_pde + w_dbc * L_dbc + w_ic * L_ic # one number
```

- **every term** — zero at the exact solution

WHERE EACH TERM IS EVALUATED

$\mathcal{L}_{data}$ measurements, if any

$\mathcal{L}_{PDE}$ interior collocation points

$\mathcal{L}_{DBC}$ values on the boundary

$\mathcal{L}_{NBC}$ derivatives on the boundary

$\mathcal{L}_{IC}$ the field at $t = 0$

$\mathcal{L}_{IV}$ its rate at $t = 0$

Soft Enforcement

Soft enforcement — a condition imposed as a penalty term in the loss: the network is asked to meet it at sampled boundary or initial points, competing with the PDE term for the same weights, so it holds only approximately. Neumann and initial conditions are added the same way.

THE CONDITION AS A PENALTY

$$\mathcal{L}_{DBC} = \frac{1}{n_{DBC}} \sum_{i=1}^{n_{DBC}} [u_{\theta}(\mathbf{x}_i, t_i) - \bar{u}(\mathbf{x}_i, t_i)]^2 \quad \mathcal{L}_{NBC} = \frac{1}{n_{NBC}} \sum_{i=1}^{n_{NBC}} [\partial_n u_{\theta}(\mathbf{x}_i, t_i) - \bar{q}(\mathbf{x}_i, t_i)]^2$$

the given value $\bar{u}$ (Dirichlet) and flux $\bar{q}$ (Neumann) against the network, on boundary points

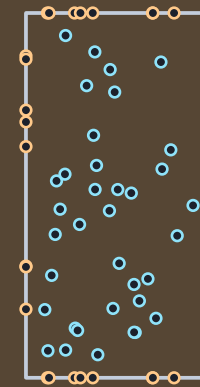
- **it competes** — with the PDE term, for the same weights
- **met approximately** — its loss weight sets how closely

SOFT WALLS IN PYTORCH

```
L_dbc = (net(x_wall) - u_bar).pow(2).mean() # the value, sampled
L_nbc = (dudn(net, x_flux) - q_bar).pow(2).mean() # the flux, sampled
loss = L_pde + w_dbc * L_dbc + w_nbc * L_nbc # requests, at rates w
```

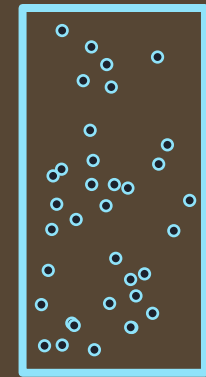
THE SAME WALL, ASKED OR BUILT IN

SOFT



$loss + w \cdot (u - \bar{u})^2$
at the wall: $u \approx \bar{u}$

HARD



$u = A + D \cdot N$
at the wall: $D = 0, u = \bar{u}$

Soft asks for the wall at the amber points and meets it nearly. Hard multiplies the network by D , zero on every wall, and meets it exactly.

Hard Enforcement

Hard enforcement — a plain network is $u = \mathcal{N}(x; \theta)$; hard enforcement writes the trial solution $u = A(x) + D(x) \cdot \mathcal{N}(x; \theta)$. A has the wall value and D is zero on the wall, so every θ meets it exactly, and training picks the best candidate. The price: a new D per geometry.

THE CONDITION BUILT INTO THE NETWORK

$$u_{\text{trial}} = A + D \mathcal{N}_{\theta}, \quad D(x, y) = x(1-x)y(1-y), \quad A = 0$$

the mask D for the unit square: zero on the edges $x = 0, 1$ and $y = 0, 1$, positive inside

- D — a mask: zero on the wall, free inside
- the boundary term — gone, nothing left to balance
- the rule — soft by default, hard when D is easy

TALK · 2 MIN How would you explain the difference between a soft and a hard condition, in your own words? Then, in the table on the right, why is the hard network more accurate inside the square as well as on its walls?

MEASURED · POISSON, UNIT SQUARE

	soft, $w = 1$	soft, $w = 100$	hard
total error $\ u_{\theta} - u\ / \ u\ $	9.9e-3	7.0e-4	3.2e-5
wall error $\max u_{\theta} $	2.2e-2	1.5e-3	0
PDE residual, mean r^2	3.8e-4	7.2e-4	6.5e-5

Exact $u = \sin \pi x \cdot \sin \pi y$. Total error: the miss over the square relative to u ($1e-2 = 1\%$); wall error: the largest miss on the walls, where $u = 0$; PDE residual: the mean of r^2 inside. 101×101 test grid; $\tanh 40 \times 2$, 320 interior, 240 boundary, seed 0.

A HARD WALL IN PYTORCH

```
def u(xy):
    x, y = xy[:, :1], xy[:, 1:]
    return x*(1-x)*y*(1-y)*net(xy)
loss = r(u, xy).pow(2).mean()
```

the trial solution
the coordinates
D times N, A = 0
the PDE term only

Loss Weight Rebalance

Loss weight rebalance — choosing the weights w_k in $\mathcal{L} = \mathcal{L}_{PDE} + \sum w_k \mathcal{L}_k$ so that every term pulls on θ as hard as the equation does. A term with a small gradient is neglected by the optimiser, and its condition is met only roughly; rebalancing raises its weight until it counts.

BY HAND: THREE GUIDELINES

- **scale first** — x, u and f of order one
- **start at $w_k = 1$** — raise the term that stalls
- **sweep 1, 10, 100** — keep the lowest test error

AUTOMATIC: FROM GRADIENT NORMS

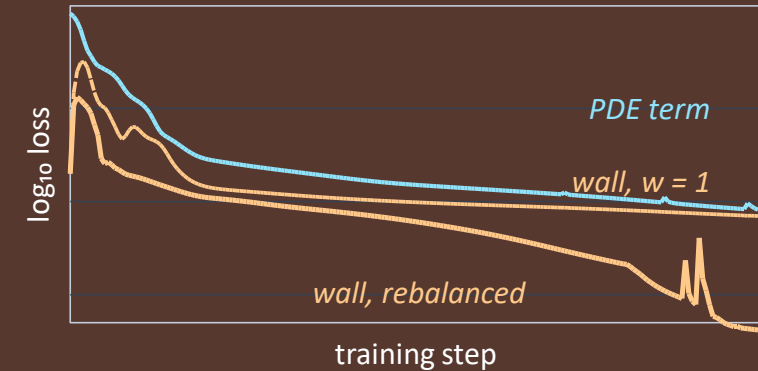
$$\hat{w}_k = \frac{\|\nabla_{\theta} \mathcal{L}_{PDE}\|}{\|\nabla_{\theta} \mathcal{L}_k\|} \quad w_k \leftarrow 0.9 w_k + 0.1 \hat{w}_k$$

$\|\nabla_{\theta} \mathcal{L}_k\|$: how hard term k pulls on θ ; the average keeps w_k steady

REBALANCING IN PYTORCH

```
g_pde = grad(L_pde, theta, retain_graph=True) # pull of the PDE term
g_dbc = grad(L_dbc, theta, retain_graph=True) # pull of the wall term
w_dbc = 0.9*w_dbc + 0.1*norm(g_pde)/norm(g_dbc) # raise the wall
```

MEASURED · LOSS PER TERM



	$w = 1$	$w = 10$	$w = 100$	rebalanced
total error	9.9e-3	1.6e-3	7.0e-4	1.3e-3
wall error	2.2e-2	3.6e-3	1.5e-3	3.5e-3

At $w = 1$ the wall loss stalls; rebalanced, w rose to about 10 and the wall loss kept falling, near the best hand weight without a sweep.

Collocation

Collocation — evaluating the residual at a finite set of sampled points, the collocation points, in the domain and on its boundary. Between the points nothing is imposed, so their number and placement set the accuracy; the interior needs at least N^* of them.

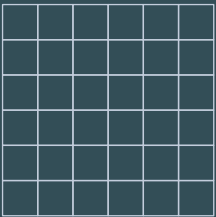
HOW MANY POINTS: AT LEAST N^*

course: $N^* = (D_i + 1) + (D + 1) \times K$
 D_i inputs, K hidden layers of D neurons; each $+1$ is a bias
Liu: $P^* = (p + 1) + (N_n + 1) \times N_L$

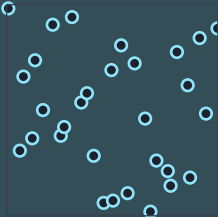
- N^* — one condition per neuron and bias, not per weight
- **in practice** — 10 to $20 \times N^*$, Ex_07.1 uses 15

	FDM node	collocation point
carries	its own unknown value	no unknown: the weights are shared
how many	error $\propto h^2$: refine to the accuracy	at least N^* , counted from the network
grows with	the mesh: $1 / h^2$	D neurons, K layers, D_i inputs
Ex_07.1	$41 \times 81 = 3\,081$ nodes, one current	$2\,000 = 15\, N^*$, all currents

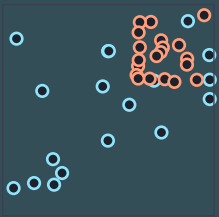
THREE WAYS TO PLACE THE POINTS



uniform grid



Latin hypercube



adaptive

N^* , POINTS AND FDM NODES · 3 INPUTS

network	N^*	points, $15\, N^*$	weights	FDM, as many
32×4	136	2 040	3 329	41×81
100×2	206	3 090	10 601	73×145
200×4	808	12 120	121 601	247×493

32×4 : $N^* = (3 + 1) + (32 + 1) \times 4 = 136$. Points grow with the neurons, weights with their square. Orange: points added by refinement.

Advanced Topics

RELATED TOPICS LEFT OUT, AND WHERE THEY MATTER

topic	what it adds	read	where it makes a difference
energy method	an energy, first derivatives only	Liu ch. 2.8	elasticity, structures
the stencil	finite differences on the network	Liu ch. 2.7.6	costly derivatives
learned boundary	a second network for curved walls	Liu ch. 3.1 – 3.3	holes, curved geometry
adaptive sampling	points where the residual is large	Liu ch. 5.6	shocks, thin layers
periodic activations	sine networks for oscillations	Liu ch. 5.7.4	waves, vibration

WHY THEY ARE LEFT OUT

Each answers a difficulty the problems of this block do not meet. A plain PINN, with soft or hard conditions, solves Ex_07.1; the rest return when a problem asks for them.

Exercise 7.1

ONE NOTEBOOK, SECTION BY SECTION

- **1** — the slot and the heat its winding makes
- **2** — the exact and the finite-difference solution
- **3** — the mesh density for an agreed 0.01 K
- **4 – 5** — a PINN, soft and hard, and the three compared
- **6 – 7** — the report; mini projects MP7.1A and MP7.1B

THE QUESTION EACH PART ANSWERS

- 1 – 2** how hot is it, exactly?
- 3** which mesh is fine enough?
- 4 – 5** soft or hard, and against FDM?

REPORT QUESTION, TO CONCLUDE

Finite differences met 0.01 K for almost nothing. For which problem would you train a physics-informed network instead, and what would you check it against?

Key Takeaways

PINN

what it is	a network that is the solution field $u(x, t)$
function	coordinates in, u out; derivatives by autograd
train, tune	tanh, at least N^* points, Adam then L-BFGS
vs FEM	weights, not nodal values; no mesh
used for	forward and inverse problems, all of Part 2

PINN LOSS

what it is	one mean squared residual per equation
function	soft: each condition asked at its own points
train, tune	weights as rates of exchange; balance them
vs soft	hard: exact on the wall, and more accurate
used for	every block of Part 2, a different residual

Next — L7.2: the fundamental equations, and the conditions each one needs.

Questions

1 – 5 · PINN

- 1 What is a physics-informed network: what does it take in and give out, and how does its loss differ from a supervised one? *Liu ch. 2.1*
- 2 What function does it compute: what does the ansatz promise, and how many collocation points does N^* ask for? *Liu ch. 2.1, 2.5.2; Ex_07.1 section 4*
- 3 How is it trained and tuned: why must the activation be smooth to the order of the equation, and what does ReLU do? *Liu ch. 4.7*
- 4 How does it differ from a mesh-based solver: what are the unknowns, and what takes the place of the mesh? *Liu ch. 1 – 2.1*
- 5 What is it used for: what is the operator form, which conditions make it well posed, and what is an inverse problem? *Liu ch. 2.2; Ex_07.1 sections 1 – 2*

6 – 10 · PINN LOSS

- 6 What is the composite loss: which terms does it have, and where is each one evaluated? *Liu ch. 2.2.2*
- 7 How does soft enforcement impose a condition, and why is it met only approximately? *Liu ch. 2.2.2; Ex_07.1 section 4*
- 8 How is the loss tuned: what is a loss weight, physically, and what does gradient-norm balancing do? *Liu ch. 2.2.3; Ex_07.1 section 4*
- 9 How does hard enforcement differ from soft: what does the trial solution guarantee, and what does a hole cost? *Liu ch. 2.5.3, 2.9; Ex_07.1 section 4*
- 10 What is the loss used for across Part 2: what changes from block to block, and where are the collocation points placed? *Liu ch. 2.6; Ex_07.1 section 4*

NEXT

L7.2 — Fundamental PDEs

The same loop for every equation: only the residual and its conditions change.

BEFORE THE NEXT SESSION

read	Liu ch. 2.1 – 2.9
run	Ex_07.1, sections 1 to 5
bring	the wall error, soft and hard
think about	which conditions your problem carries

Part 2 has one method and many applications. This was the method.